Closures: functions that remember

A function made inside another call keeps that call's variables alive, so it can use them, and change them, later.

THE QUESTION

How can a function remember a value after the call that made it has ended?

THE INTUITION

A `def` inside a function makes a new function object on every call. If that inner function uses the call's variables, Python keeps them alive for as long as the inner function exists.

KEY TAKEAWAY

A closure is a function plus the variables of the call that made it. Each call makes a fresh set; `nonlocal` lets the inner function change them.

1 A `def` inside a `def` stays inside

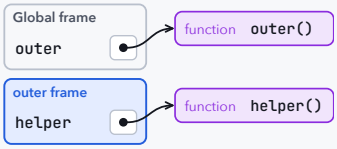
nested.py

```
1 def outer():
2     def helper():
3         return "inside"
4     return helper() + "!"
5
6 print(outer())
7 print(helper())
```

prints `inside!`

then `NameError: name 'helper' is not defined. Did you mean: 'help'?`

while outer() runs line 4 (sys.settrace)



after the call: the outer frame is gone, and helper with it; Global has only outer

A **nested function** is defined inside another, its **enclosing** function. The inner `def` runs at every call of `outer` and makes `helper` a local name of that call (the functions sheet). When the call returns, its frame and `helper` are gone: outside, a **NameError** (its "Did you mean" guess is wrong).

2 Return the function; do not call it

multiplier.py

```
1 def make_multiplier(n):
2     def times(x):
3         return x * n
4     return times
5
6 triple = make_multiplier(3)
7 print(triple)
8 print(triple(10))
```

line 7 `<function make_multiplier.<locals>.times at ...>`

line 8 `30`

number in, function out



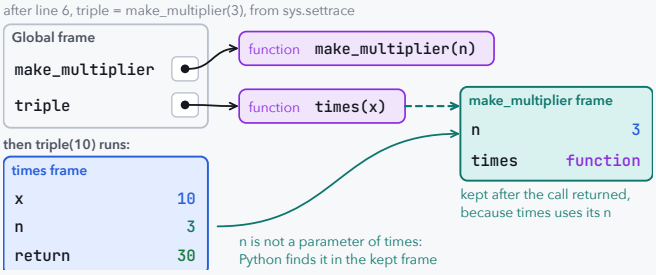
with line 4 as return times():

```
TypeError: make_multiplier.<locals>
.times() missing 1
required positional argument: 'x'
```

`make_multiplier` multiplies nothing: it defines `times` and returns that function object, with no brackets. Each call gives a function that multiplies by its own `n`, as `sqrt_f` does on the functions-are-values sheet. `return times()` would call `times` there, with no `x`.

3 The inner function keeps its call's frame alive

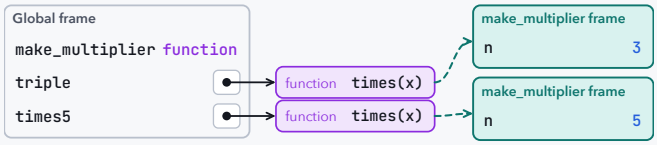
after line 6, triple = make_multiplier(3), from sys.settrace



kept after the call returned, because times uses its n

A frame normally vanishes at `return`. But `times` uses `n` from the enclosing call, a **free variable**, so Python keeps it. A function plus the variables it keeps is a **closure**. `triple(10)` gets a new frame with `x = 10` and finds `n = 3` in the kept one (lookup order: the scope sheet).

4 Each call makes a new, separate frame



two calls, two kept frames

```
triple(10), times5(10)              → 30 50
triple.__closure__[0].cell_contents → 3
triple.__code__ is times5.__code__  → True
```

A second call runs the inner `def` again: a new function with its own `n = 5`, sharing the code (`__code__`). `f.__closure__` is a tuple of **cells**, one per free variable, and `cell_contents` is the value in one (5 for `times5`). Printing a cell shows only `<cell at ...: int object at ...>`.

5 To change a kept variable, say `nonlocal`

counter.py

```
1 def make_counter():
2     count = 0
3     def step():
4         nonlocal count
5         count = count + 1
6         return count
7     return step
```

each call, from sys.settrace

call	count before	after	returns
tick()	0	1	1
tick()	1	2	2
tick()	2	3	3
other()	0	1	1
tick()	3	4	4

amber: other = make_counter(), a new frame

without line 4 (nonlocal count), the first tick() stops with

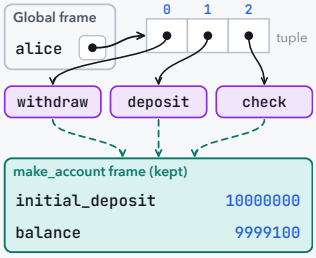
```
UnboundLocalError: cannot access local variable 'count'
where it is not associated with a value
```

Reading `count` needs nothing extra, but `count = count + 1` assigns, and that makes `count` local to `step` (the scope sheet), with no value yet. `nonlocal count` says the name belongs to the enclosing call. Each `tick()` adds 1 in its kept frame; `other` counts on its own.

6 Three functions can share one kept variable

the notebook's make_account

```
1  def make_account(initial_deposit):
2      assert initial_deposit >= 0
3      balance = initial_deposit
4      def withdraw(amount):
5          nonlocal balance
6          assert balance >= amount
7          balance = balance - amount
8      def deposit(amount):
9          nonlocal balance
10         balance = balance + amount
11     def check():
12         nonlocal balance
13         return balance
14     return (withdraw, deposit, check)
```



after alice[0](1000) and alice[1](100)

The notebook's bank account: one call, one frame, three functions returned as a **tuple** (`alice[0]` is `withdraw`). All three use the same `balance`, as the notes draw with three arrows to one box: `alice[2]()` gives 9999100. `check` only reads, so its `nonlocal` could go.

Accounts, loops and a password generator

Separate accounts from one maker, the loop that gives every function the same number, and a password generator.

7 Every account has its own balance

you run, line by line	alice balance	bob balance	shown
`alice = make_account(10000000)`	10000000		
`alice[0](1000)`	9999000		
`alice[1](100)`	9999100		
`bob = make_account(1000)`	9999100	1000	
`bob[0](50)`	9999100	950	
`bob[1](1000)`	9999100	1950	
`alice[2]()`	9999100	1950	9999100
`bob[0](2000)`	9999100	1950	AssertionError
`carol = make_account(-100)`	9999100	1950	AssertionError

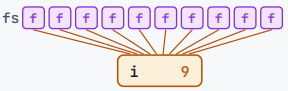
balances read from each account's kept frame; blue: changed by this line

`bob` gets a new frame, so his calls never touch `alice`'s balance. An `assert` stops with an **AssertionError** when its condition is False: taking 2000 from 1950 fails on line 6, before the subtraction; `make_account(-100)` fails on line 2, so no `carol`.

8 A loop's functions all see its last value

the notebook's loop

```
1 fs = []
2 for i in range(10):
3     def f(x):
4         return i + x
5     fs.append(f)
6
7 for f in fs:
8     print(f(1))
```



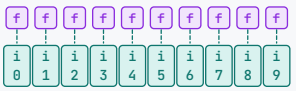
all ten read the one global i, left at 9

prints `10 10 10 10 10 10 10 10 10 10`

the homework: one call per value

```
1 def make_adder(i):
2     def f(x):
3         return i + x
4     return f
5 fs = []
6 for i in range(10):
7     fs.append(make_adder(i))
```

then the same for f in fs: print(f(1))



each f keeps its own make_adder frame

prints `1 2 3 4 5 6 7 8 9 10`

`f` reads `i` only when called. All ten share one global `i`, left at 9, so each `f(1)` is 10; the notebook's `j = i` inside `f` runs too late as well. The fix makes each `f` in its own call, whose frame keeps that `i`; `def f(x, i=i):` also works.

9 The password generator, one letter at a time

random.seed(1), then make_password("csldx"), pass by pass (sys.settrace)

pass	c	its branch picks one of	picked	p after
1	c	A to Z	E	'E'
2	s	a to z	s	'Es'
3	l	A to Z or a to z	z	'Esz'
4	d	0 to 9	1	'Esz1'
5	x	`~!@#$%^&*()_-+=/?.>,<	&	'Esz1&'

```
make_password("dddd")      → '1777'
make_password("ccssccxxdx") → 'UMzgDP`_6>'
```

next in the same sequence

Per format letter, an `if`/`elif` branch adds `random.choice(pool)`: one character of the string, picked at random. `random.seed(1)` fixes where the random sequence starts, so these outputs repeat every run; drop it in real use, and prefer the `secrets` module for real passwords.

10 Choices as data; the format in a closure

the notebook's dictionary

```
possibilities = {
    "c": "ABCDEFGHIJKLMNOPQRSTUVWXYZ",
    "s": "abcdefghijklmnopqrstuvwxyz",
    "l": "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
         "abcdefghijklmnopqrstuvwxyz",
    "d": "0123456789",
    "x": "`~!@#$%^&*()_-+=/?.>,<",
}
```

grey: two strings side by side join into one

ours: a closure that keeps the format

```
def password_maker(fmt):
    def make():
        return make_password(fmt)
    return make
```

```
new_pin = password_maker("dddd")
new_pin() → '2914'
new_pin() → '1777'
```

its cell holds `'dddd'`

"Move complexity from code to data": a **dictionary** (the dictionaries sheet) maps each letter to its string, so the five branches become `p = p + random.choice(possibilities[c])` (same seed, same output). Our `password_maker` keeps `fmt` in a closure.

11 Common mistakes, with `make_multiplier`, `make_counter` and `make_account` as above

you wrote	the last line Python prints, or what you see; why, and the fix
`triple = make_multiplier(3)(10)` `triple(2)`	`TypeError: 'int' object is not callable` `(10)` already called it, so `triple` is 30; `make_multiplier(3)(2)` gives 6
`print(make_counter()(),` `make_counter()())`	`prints 1 1` a new `count` per call; one `tick = make_counter()` called twice gives `1 2`
`count = 0` `def step(): nonlocal count ...`	`SyntaxError: no binding for nonlocal 'count' found` `nonlocal` needs a variable of an enclosing function, as in panel 5
`alice = make_account(100)` `print(alice[2])`	`prints <function make_account.<locals>.check at ...>` `alice[2]` is the check function itself; call it: `alice[2]()` gives 100

12 Try it (answers in green)

1. `make_adder(n)` returns `add(x)`, giving `x + n`. `make_adder(2)(5)`?
   7
2. Counters `a` and `b`. After `a()`, `a()`, what does `print(a(), b())` show?
   `3 1`: two frames.
3. `make_power(k)` returns `power(x)`, giving `x ** k`. `make_power(3)(2)`?
   8
4. `fs` holds `lambda: k` for each `k in range(3)`. `[f() for f in fs]`?
   `[2, 2, 2]`; with `k=k`: `[0, 1, 2]`
5. `add = make_total()` keeps a running total. `add(5)`, then `add(10)`?
   15, with `nonlocal total` in `add`
6. Write `make_greeter(word)`: `make_greeter("Hi")("Bob")` is `'Hi, Bob!'`.
   inner `greet(name)` joins `word, ", ", name, "!"`

a def inside a def: a new function per call · return it uncalled · it keeps the call's variables · `nonlocal` changes them

Source: the closures notebook (bank account, functions made in a loop) with its handwritten notes, and the password generator notebook, nipunbatra.github.io/comp22; names changed; make_multiplier, make_counter, make_adder and password_maker are ours. Outputs and errors come from Python 3.11.11, frames from sys.settrace (checked against __closure__), addresses shown as "..."; password runs start with random.seed(1), so they repeat. Next: the readable code and sets sheet.